

UGTS-KC CHESS 1.0

Proof-Carrying Deterministic Solver

Exact rule kernel, hash-chained replay, finite-horizon mate certificates, alpha-beta search and complete KQK/KRK tablebases

UGTS-KC CHESS 1.0 - PROOF-CARRYING SOLVER ARCHITECTURE

Authoritative chess state and verified moves remain upstream of search, proofs and presentation.

SUPPLIED SUBSTRATE PRINCIPLES



AUTHORITATIVE CHESS SUBSTRATE



EXACT / HEURISTIC / DEPLOYMENT LAYERS



FEN - legal moves - check/mate castling - en-passant - promotion OR/AND finite-horizon mate certificates independent verifier iterative deepening alpha-beta quiescence - transposition table complete KQK and KRK WDL + DTM retrograde tables

OUTPUTS



Prepared for	Tom Klootwijk
Document date	29 August 2026
Version / schema	1.0.0 / ughts-kc-chess-solver-1.0
Release status	Exact bounded solver substrate with executable validation

EVIDENCE BOUNDARY

The package does not claim a weak or strong game-theoretic solution of the standard initial position. It supplies an exact rules kernel, exact bounded proofs, exact three-piece tablebases and a specified architecture for expansion.

0. Contents and Executive Decision

The word “solve” is split into exact rules, exact bounded claims, heuristic search and an uncompleted global proof program.

1 Source grounding and scope	2 What “solve chess” means
3 Formal state and move authority	4 Rule kernel and terminal semantics
5 Deterministic events, hashes and replay	6 Heuristic search layer
7 Proof-carrying bounded mate solver	8 Complete KQK/KRK tablebases
9 Global-solve architecture	10 Implementation and command line
11 Validation and release evidence	12 Mechanism catalog
13 Boundaries, kill criteria and final definition	Appendix A Package map and reproduction
Appendix B Source register and references	

Executive decision

GO - as a bounded proof-carrying solver substrate. Standard chess can be represented cleanly in the UGTS chain because every move is a finite proposal with explicit support, compatibility, guard and transition conditions. The implementation is intentionally split so that legality and proof verification remain exact, while general position scoring remains visibly heuristic.

Table 1. Release interpretation

Layer	Status / meaning
Exact rule authority	FEN parsing; legal move generation; check, checkmate, stalemate; castling, en-passant and promotion; deterministic transitions.
Exact bounded solving	OR/AND forced-mate certificates within a declared ply horizon, independently replayed and verified.
Exact finite state spaces	Complete KQK and KRK win/draw/loss plus depth-to-mate over all 2^19 dense addresses per material class.
Heuristic search	Iterative deepening alpha-beta, quiescence and a transposition table. Non-mate scores are estimates.
Global chess solution	A certificate-sharded architecture is specified. The standard initial position is not claimed solved.
RELEASE RESULT 65 automated tests pass; 19 perft checks match; two JSON schemas validate; the mate certificate verifies; two exact tablebases load with matching SHA-256 hashes; and the installable wheel builds successfully.	

1. Source Grounding and Scope

The chess profile preserves the supplied documents' terminology, operation order and evidence boundaries; it does not silently import unsupported claims.

Source Grounding: Suitable UGTS Contributions

UGTS-GN 1.1	CORE	support -> compatibility -> guard -> event -> transition -> lineage; packed-state and evidence boundaries
UGTS-KC 2.0	CORE	typed patterns and kinematics; hybrid event degeneracy and explicit error contracts
KC Two Hands 3.0	CORE	stable identity; proposal verification; deterministic commit; checkpoints and replay
UGTS-KC 3.6	CORE	definitions as content-addressed data; dependency closure and explicit operation order
UGTS-KC 3.6.2 SCLP	CORE+	finite key layouts; guard intervals; bounded grammar and storage audits
UGTS-KC 3.9	ADDITIVE	deterministic game runtime; browser-facing proof viewer and compact project records
UGTS-KC 3.6.3 SARA	EXCLUDED	wallet-specific public/secret and authorization profile; not needed for chess mechanics

Engineering decision: preserve terminology and boundaries; do not import claims that the sources do not establish.

Figure 1. Suitable source contributions. SARA is reviewed but excluded as domain-specific rather than forced into chess mechanics.

Table 2. Source-role register

Supplied source	Status	Chess-profile contribution
UGTS-GN 1.1	CORE	Canonical support, compatibility, guard, event, transition, lineage; precision and compression boundaries
UGTS-KC 2.0	CORE	Typed state, query-first patterns, topology, hybrid events, bounded dynamics
KC Two Hands 3.0	CORE	Proposal verification, deterministic commit, state hashes, checkpoints and replay
UGTS-KC 3.6	CORE	Definitions as data, content addresses, dependency closure and explicit operation order
UGTS-KC 3.6.2 SCLP	CORE	Finite packed keys, radix tries, bounded branching and storage audits
UGTS-KC 3.9	ADDITIVE	Deterministic game world, canonical snapshots/hashes, CLI and offline HTML5 delivery
UGTS-KC 3.6.3 SARA	EXCLUDED	Reviewed but excluded as wallet/security-specific; only its explicit authorization boundary is retained as a governance analogy

Source grounding: UGTS-GN 1.1 supplies the canonical support -> compatibility -> guard -> event -> transition -> lineage order and the rule that compression/one-bit records never imply complete state. KC 2.0 contributes typed kinematics and hybrid event discipline. Two Hands 3.0 contributes proposal verification, deterministic commit and replay. KC 3.6 contributes content-addressed definitions and explicit dependency order. SCLP 3.6.2 contributes finite key and guard-interval discipline. KC 3.9 contributes deterministic runtime and self-contained browser-delivery patterns.

2. What “Solve Chess” Means

A precise release must separate complete claims from estimates and future architecture.

Three Meanings of “Solve” in This Package

Exact now	Heuristic now	Global solve architecture
✓ legal move relation ✓ mate/stalemate and bounded draw policy ✓ finite-horizon forced-mate certificates ✓ complete KQK/KRK WDL + DTM ✓ hash-checked replay	✓ iterative-deepening alpha-beta ✓ quiescence search ✓ material and piece-square evaluation ✓ transposition table ✓ principal-variation reporting	✓ content-addressed position shards ✓ certificate-first worker outputs ✓ retrograde / proof frontier expansion ✓ conflict and hash reconciliation ✓ future compute, not claimed complete

Non-negotiable boundary: a search score is not a proof; an unfinished global architecture is not a solution of the initial position.

Figure 2. Three meanings of solve. Only the left column is exact in the current release; the middle is heuristic; the right is a future proof program.

2.1 Exactness classes

- Rule exactness: a legal-move answer is binary and independently reproducible from the same position and history context.
- Certificate exactness: a proved mate requires one attacker choice at every OR node, full defender coverage at every AND node and a verified checkmate leaf.
- Tablebase exactness: every legal state in the declared KQK or KRK material class receives a WDL and, for decisive states, a depth-to-mate value.
- Search estimate: a centipawn score is a bounded computation result, not a theorem about perfect play.
- Unknown is valid: failure to find a proof within a horizon means only `not_forced_within_horizon`.

2.2 Relation to public tablebase context

The public Syzygy ecosystem currently provides exact play for positions with up to seven pieces. This package does not reproduce that corpus. It deliberately implements two compact three-piece DTM tables so that the complete generation, storage, probing, hashing and validation path is inspectable inside the accompanying ZIP.

NON-CLAIM

The standard 32-piece initial position is not weakly solved, strongly solved or ultra-weakly solved by this package. “UGTS Chess 1.0” names a solver substrate and release profile, not a declaration that all chess has been computed.

3. Formal State and Move Authority

The authoritative object is a typed position plus history and lineage, not an image of a board and not a search score.

3.1 Typed position

```
q = (B, s, C, e, h, n, H, L)

B : 64 typed board cells
s : side to move
C : four castling-right flags
e : optional en-passant target
h : halfmove clock
n : fullmove number
H : repetition-history context
L : ordered event lineage
```

Identity split. Board coordinate, piece type, side-to-move, castling rights, en-passant target, clocks and lineage remain distinct. The one-bit side field has a narrow selector role; it never replaces the rest of the state. FEN is the portable position spelling, while history is supplied separately when repetition adjudication is required.

Canonical UGTS Chess Move Authority

Search may propose. Only this ordered verifier changes authoritative state.



Figure 3. The exact move-authority sequence. Search and user input can propose moves, but neither may bypass the verifier.

3.2 Verified move condition

```
legal(q, m) = in_support(q, m)
              and compatible(q, m)
              and special_guards(q, m)
              and not king_attacked(apply(q, m), mover)
```

The king is never captured. A checkmate terminal is defined as check plus zero legal replies. This keeps the transition semantics aligned with the rule system instead of allowing an illegal king-capture patch.

4. Rule Kernel and Terminal Semantics

The dependency-free move generator implements standard piece movement and the full state patches needed by special moves.

4.1 Movement support and compatibility

Table 3. Piece support and guards

Piece	Finite support	Compatibility / guard
Pawn	One step; optional initial two-step; diagonal capture; four promotion choices; en-passant target test.	Direction, occupancy, promotion rank, captured pawn and king safety.
Knight	Eight finite leaper offsets.	Board bounds, own occupancy and king safety.
Bishop	Four diagonal rays until first blocker.	Ray occupancy, enemy non-king capture and king safety.
Rook	Four orthogonal rays until first blocker.	Ray occupancy, enemy non-king capture and king safety.
Queen	Bishop plus rook rays.	Same ray and king-safety rules.
King	Eight adjacent offsets plus castling proposals.	No king adjacency, no attacked destination; castling path and rights guards.

4.2 Special move patches

- Castling: the rook must exist on its home square; the path must be empty; the king may not start in, pass through or finish in check; the king and rook are patched atomically.
- En-passant: the target square is transient; the captured pawn is removed from a different square; legality is tested after the full patch so discovered attacks on the king are rejected.
- Promotion: four typed outcomes (queen, rook, bishop or knight) are emitted as distinct moves; capture and promotion flags may coexist.
- Rights and clocks: king/rook movement and home-rook capture clear the appropriate castling flags; pawn moves and captures reset the halfmove clock; Black moves advance the fullmove number.

4.3 Terminal and draw status

Table 4. Implemented terminal and draw policy

Status	Authority	Implementation condition
Checkmate	Terminal, decisive	Side to move is in check and has zero legal moves.
Stalemate	Terminal draw	Side to move is not in check and has zero legal moves.
Dead-position subset	Terminal draw	Exact conservative subset: kings only; king plus one bishop/knight; bishops only on one square color.
Fivefold repetition	Automatic draw	History key occurs at least five times.
75-move rule	Automatic draw	Halfmove clock reaches 150 plies.
Threefold repetition	Claimable draw	History key occurs at least three times when claim policy is enabled.
50-move rule	Claimable draw	Halfmove clock reaches 100 plies when claim policy is enabled.

BOUNDARY

The dead-position detector is deliberately conservative and exact for its implemented subset; it is not advertised as a complete theorem prover for every possible no-mate material configuration.

5. Deterministic Events, Hashes and Replay

Moves are proposed in parallel or by any front end, but state mutation occurs through one ordered commit boundary.

5.1 Proposal record

```
MoveProposal = {
  proposal_id, source, parent_hash, move_uci,
  support_ok, compatibility_ok, guard_ok,
  reason_codes, requested_patch
}
```

A legal proposal carries evidence; an illegal proposal carries reasons. The commit path recomputes legality rather than trusting a caller-provided flag. The event writes an ordered sequence number, the move, SAN, pre-state hash, post-state hash, resulting FEN and lineage label.

5.2 Canonical identities

Identity	Role and boundary
State SHA-256	Canonical full-position dictionary. Used in proofs, replay and release evidence.
Repetition key	Board, side, castling and legally relevant en-passant state. Used with an external history counter.
Compact 64-bit key	Deterministic search-table checksum. Useful for a transposition table; not treated as collision-free durable identity.
Tablebase 19-bit key	Bijjective dense address inside one declared KQK/KRK profile.

5.3 Replay invariant

1. Load the exact initial FEN and compute its state hash.
2. For each event in sequence, confirm the stored pre-hash equals the current hash.
3. Parse the stored UCI move against the current legal move set.
4. Apply the deterministic transition and confirm the stored post-hash.
5. Stop at the first mismatch and report divergence rather than inventing continuity.

Captured demonstration. The included Ruy Lopez replay JSON reconstructs the same final state from its initial FEN and hash-chained events. A test flips an event field and confirms that replay rejects the tampered lineage.

Source grounding: This layer directly follows the Two Hands 3.0 proposal/commit/checkpoint model and the KC 3.9 canonical snapshot/hash/save discipline, while keeping the chess rule kernel authoritative.

6. Heuristic Search Layer

The general engine is deterministic and inspectable, but only mate scores supported by terminal search or separate certificates receive proof meaning.

6.1 Search stack

Mechanism	Implementation
Iterative deepening	Search depth 1..N; retain the last fully completed result under an optional time limit.
Negamax alpha-beta	Symmetric minimax recursion with alpha/beta bounds and deterministic move ordering.
Quiescence	At depth zero, extend captures and promotions; when in check, search all legal evasions.
Transposition table	64-bit position key; exact/lower/upper bounds; mate-distance normalization; deterministic shallow-entry eviction.
Evaluation	Material plus transparent piece-square terms. Returned in centipawns from the side-to-move view.
Draw context	Halfmove and repetition history passed to terminal status; configured claim policy determines claimable draws.

6.2 Worked search result

```
{
  "root_fen": "8/8/8/8/8/k7/8/1QK5 w - - 0 1",
  "depth": 4,
  "score_text": "mate +2",
  "best_move": "b1b5",
  "principal_variation_san": [
    "Qb5",
    "Ka2",
    "Qb2#"
  ],
  "nodes": 1195,
  "qnodes": 796,
  "tt_hits": 132
}
```

Interpretation. The engine finds Qb5, Ka2, Qb2# and reports mate +2. The release does not promote the measured seconds or nodes per second as a general performance claim; they describe one reference run. More importantly, the same line is independently supported by the proof certificate and KQK tablebase.

SEARCH BOUNDARY

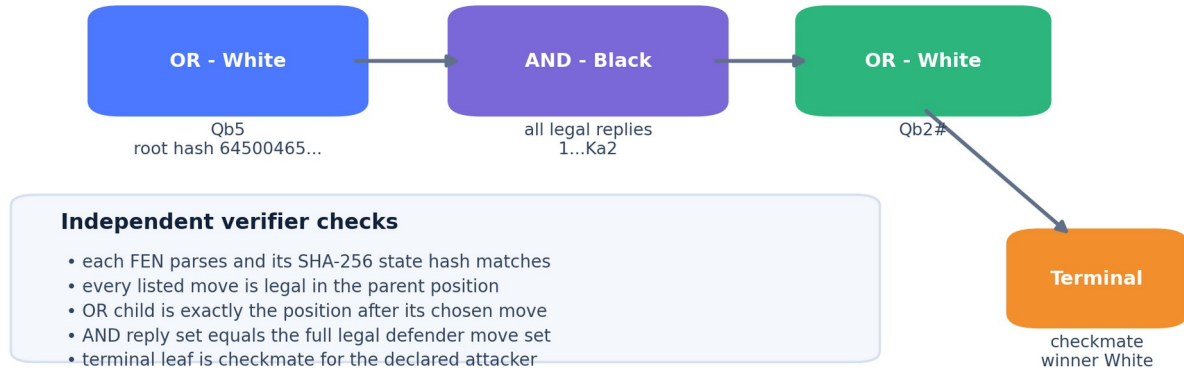
A centipawn score and principal variation may change with depth, evaluator or time limit. They are decision aids. The exact proof and tablebase interfaces use separate types and reason codes so a heuristic result cannot masquerade as a theorem.

7. Proof-Carrying Bounded Mate Solver

The certificate encodes the game-tree quantifiers, not merely one cooperative line.

Proof Certificate Semantics

OR nodes choose one proved attacker move; AND nodes enumerate every legal defender reply; leaves must verify as checkmate.



Reference certificate: 418 nodes explored by the prover; 4 certificate nodes independently verified.

Figure 4. Finite-horizon mate certificate and independent verification conditions.

7.1 Formal quantifiers

```
prove(q, attacker, p) =
  OR over legal attacker moves when side(q) = attacker
  AND over every legal defender reply otherwise

leaf success = checkmate(q) and winner(q) = attacker
```

- Each node stores its exact FEN and 64-hex SHA-256 state identity.
- An OR node stores exactly one legal attacker move and its proved child.
- An AND node stores a reply list whose UCI set must equal the complete legal defender move set.
- A terminal node must independently reconstruct as checkmate for the declared attacker.
- The verifier enforces the declared maximum ply horizon and rejects extra, missing or malformed branches.
- Repetition and the 50-move threshold are treated as proof-failure boundaries inside the path search.

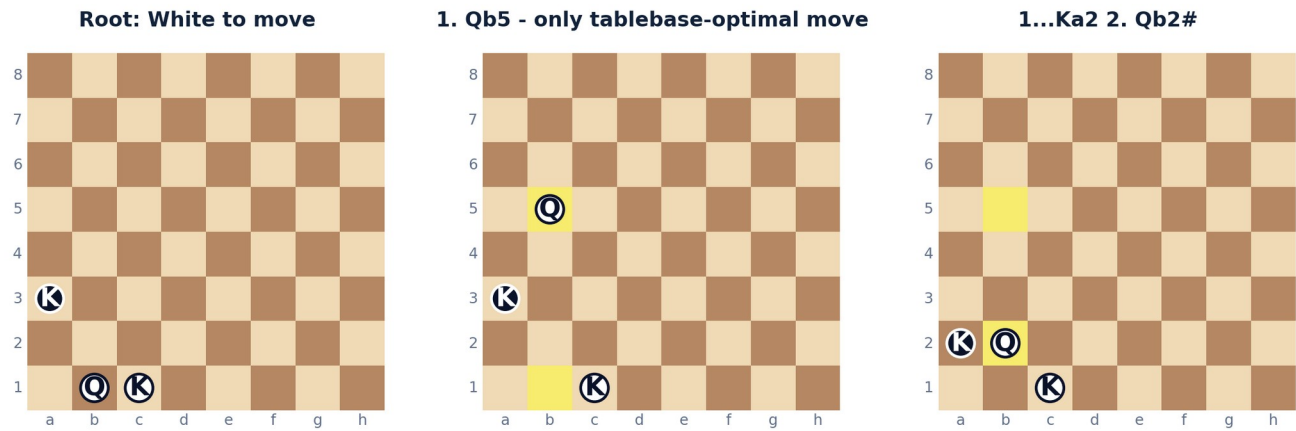
7.2 Certificate result

Field	Captured result
Root FEN	8/8/8/8/k7/8/1QK5 w - - 0 1
Attacker / horizon	White / 3 plies
Prover result	proved
Prover nodes explored	418
Certificate nodes verified	4
Forced line	1. Qb5 Ka2 2. Qb2#
Root SHA-256	64500465593f60fd69f567e84cd79634d272857edd9aad90b056123f04270b16

7.3. Worked Exact Mate in Two

Three independent mechanisms agree: legal search, the proof certificate and the KQK DTM tablebase.

Worked Exact Result: Mate in Two



FEN root: 8/8/8/8/8/k7/8/1QK5 w - - 0 1 | KQK tablebase DTM = 3 plies

Figure 5. Exact sequence from root to mate. Highlighted squares show the queen move on each transition.

Cross-checks

Independent path	Agreement
Legal kernel	Qb5, Ka2 and Qb2# parse as unique legal moves; each intermediate FEN and state hash is reproduced.
Mate prover	White has one selected winning OR branch; Black’s AND node enumerates its complete legal reply set.
Proof verifier	All four certificate nodes pass FEN, hash, legal transition, reply-coverage and terminal-mate checks.
KQK tablebase	Root is exact win with DTM 3; Qb5 leads to exact loss with DTM 2 for Black; Qb2 ends in checkmate.
Alpha-beta search	Depth 4 reports mate +2 with the same SAN principal variation.

WHY THIS IS STRONGER THAN A LINE

A displayed move sequence alone could assume cooperation. The AND certificate records every legal defender reply at each defender node. In this root there is one such reply, but the verifier checks completeness from the rule kernel rather than trusting that claim.

8. Complete KQK and KRK Tablebases

Retrograde analysis exactly classifies every valid state in each declared three-piece material space.

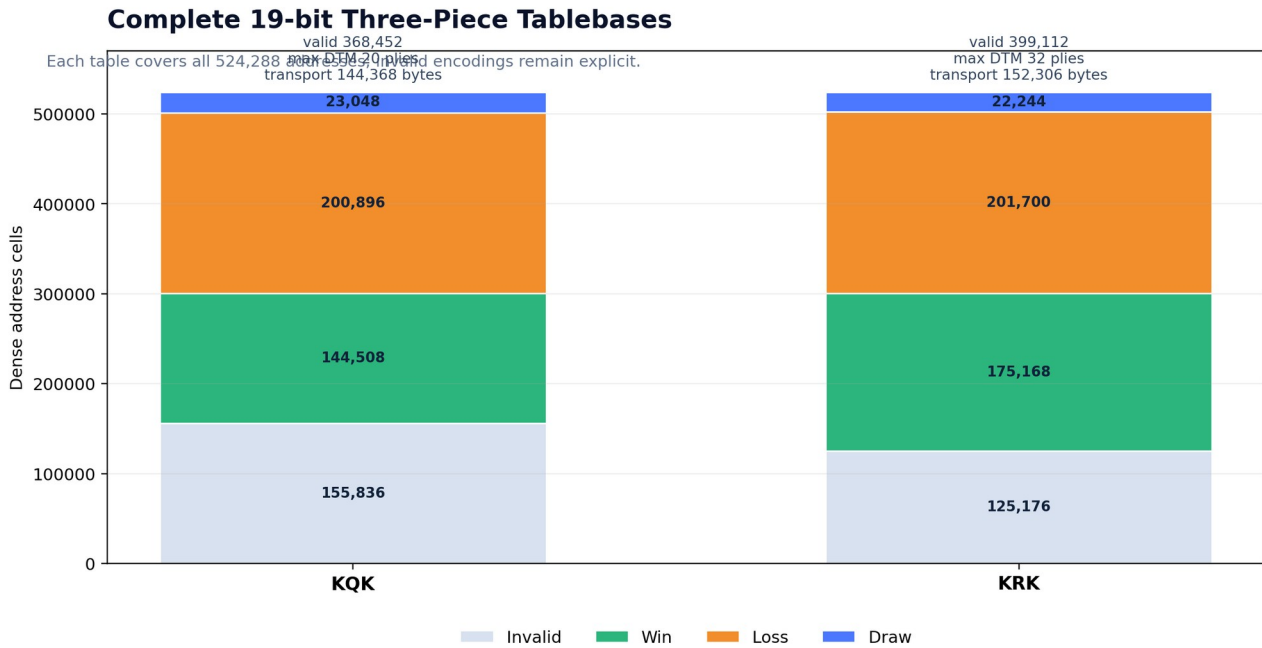


Figure 6. Dense outcome accounting. Counts sum exactly to 524,288 cells for each 19-bit table.

8.1 Dense key and normalization

```
K = (((Ks * 64) + X) * 64 + Kw) * 2 + side

Ks  strong king square 0..63
X   strong queen or rook square 0..63
Kw  weak king square 0..63
side side to move, normalized to strong-side White

address capacity = 64 * 64 * 64 * 2 = 2^19 = 524,288
```

Invalid cells remain explicit. Overlapping pieces, adjacent kings and unreachable “strong side to move while the weak king is already left in check” encodings are marked invalid instead of being silently compacted away. Black-strong positions are normalized by a 180-degree rotation and color swap.

8.2 Retrograde relation

1. Enumerate all valid cells and legal successors.
2. Seed checkmated weak-to-move states as LOSS with DTM 0 and no-move non-check states as DRAW.
3. A predecessor is WIN when it has at least one successor classified LOSS.
4. A predecessor is LOSS when every legal successor is WIN.
5. Unresolved valid cells after propagation are DRAW.
6. For WIN choose the shortest loss child; for LOSS retain the longest delaying win child.

8.3. Tablebase Metrics, Packing and Probe Contract

The compact files are finite, typed and auditable; transport compression is not confused with semantic compression.

Metric	KQK	KRK
Address cells	524,288	524,288
Valid positions	368,452	399,112
Win / loss / draw	144,508 / 200,896 / 23,048	175,168 / 201,700 / 22,244
Initial mates / stalemates	364 / 872	216 / 68
Maximum DTM	20 plies	32 plies
Raw payload	1,048,592 bytes	1,048,592 bytes
Gzip transport	144,368 bytes	152,306 bytes
SHA-256	3f38429ccd04c07f...	b52dbd40d7e6fc9b...

Probe semantics

```
{
  "material": "KQK",
  "outcome": "win",
  "dtm_plies": 3,
  "side_to_move": "white",
  "strong_side": "white",
  "exact": true,
  "best_moves": [{"move": "b1b5", "san": "Qb5"}]
}
```

- WDL is from the side-to-move perspective.
- DTM counts plies to checkmate under optimal play.
- All decisive KQK/KRK states in these tables finish within the observed maxima, below the 100-ply claim boundary.
- Each raw address stores one outcome byte and one DTM byte; gzip is only a deterministic distribution layer.
- The release stores exact SHA-256 transport hashes and tests sampled retrograde invariants.

Source grounding: This follows SCLP 3.6.2's rule that a finite key is an address into a declared profile, not the whole world, and that nominal bit-width gains are never promoted without reconstruction semantics.

9. Global-Solve Architecture

The package specifies how exact claims could scale without turning distributed search output into unchecked authority.

Scalable Global-Solve Architecture (Specified, Not Completed)

All work products are immutable, content-addressed claims with independently checkable evidence.

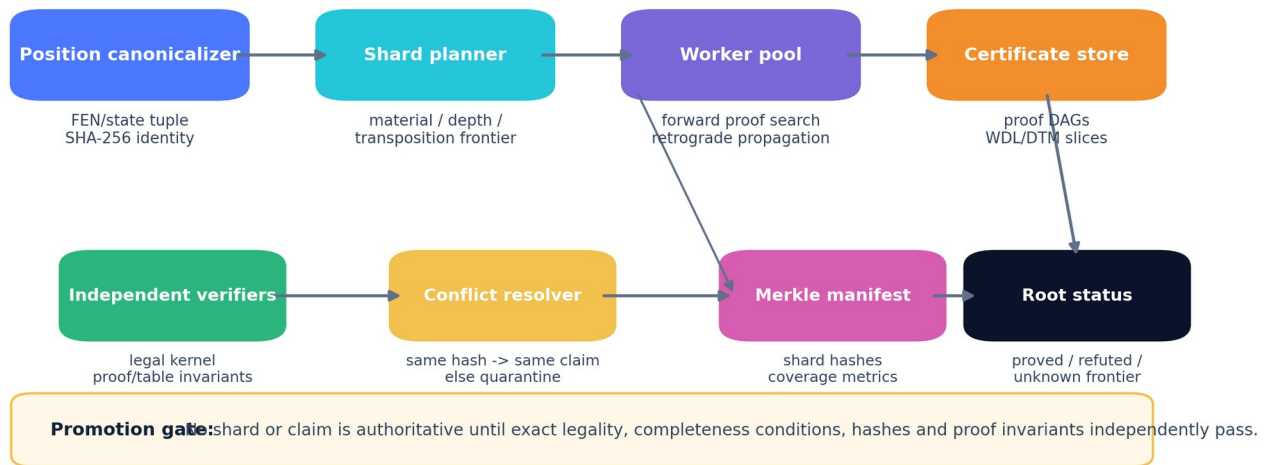


Figure 7. Certificate-first shard architecture. It is a roadmap and data contract, not evidence that the initial position is solved.

9.1 Work-unit contract

Record	Contract
Input	Canonical position identity; ruleset/profile hash; requested proof relation; resource budget; dependency shard hashes.
Worker output	Immutable claim: proved win/loss/draw, bounded non-proof, or frontier; proof DAG/table slice; metrics; content hash.
Verification	Recompute legality, terminal status, branch coverage, retrograde relations and all transport hashes in an independent process.
Conflict policy	Equal position hash and profile must yield compatible claims; contradictions are quarantined and never merged silently.
Coverage	Merkle manifest records which state sectors are exact, rejected, duplicate, missing or awaiting dependencies.
Root decision	Only a fully verified dependency closure can promote the initial position from unknown to a game-theoretic result.

9.2 Why UGTS helps

- Stable state and definition hashes make duplicated work and contradictory claims visible.
- Support and compatibility remove out-of-profile relations before expensive proof work.
- Guard and event semantics make terminal/draw conditions explicit rather than hidden in worker code.
- Lineage preserves the exact dependency path used to promote a root claim.
- Unknown and rejected are first-class statuses, preventing “not found” from being misreported as “false.”

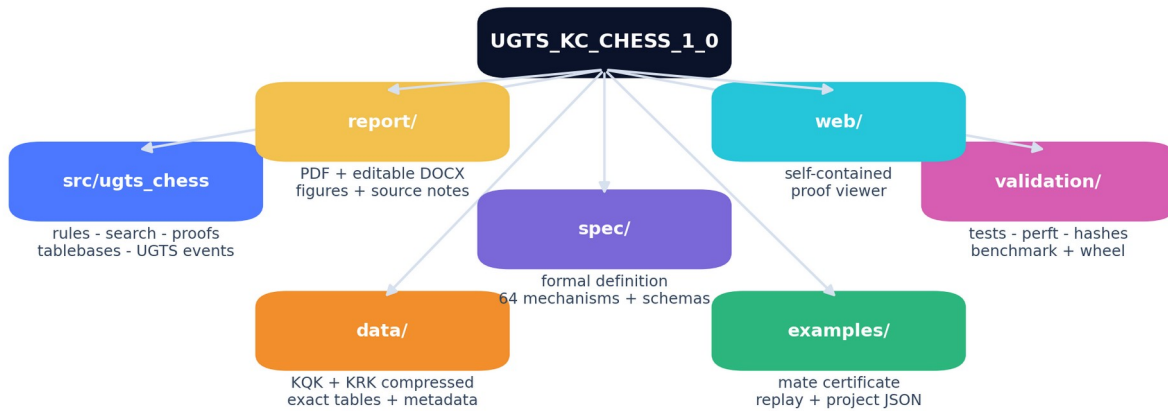
PROMOTION GATE

A larger compute cluster does not weaken the proof standard. Throughput, cache size or branch count can never replace completeness, independent verification and a closed dependency manifest.

10. Implementation and Command Line

The accompanying ZIP is self-contained, dependency-free at runtime and designed for inspection before optimization.

Accompanying Package Map



Dependency-free runtime; the generated wheel contains code and the two tablebase resources. No uploaded source PDFs are redistributed.

Figure 8. Accompanying package map. The two compressed tablebases are included both as data files and wheel resources.

10.1 Python modules

Module	Responsibility
position.py	Strict FEN parser/serializer and immutable position record.
rules.py	Attack detection, pseudo/legal moves, state transitions, SAN, terminal/draw status and perft.
hashing.py	Canonical state hash, repetition key and compact transposition key.
ugts.py	Move proposals, guard evidence, authoritative commit and hash-checked replay.
evaluate.py	Transparent material and piece-square evaluation.
search.py	Iterative-deepening alpha-beta, quiescence and deterministic transposition table.
proof.py	Finite-horizon OR/AND mate prover and independent verifier.
tablebase.py	KQK/KRK dense encoding, retrograde generation, probing and best moves.
cli.py	info, validate, perft, solve, prove-mate, verify-proof, probe, generate-tablebase and demo.

10.2. Reproduction and Examples

Every result used by this report is stored as a machine-readable artifact and can be regenerated from the package root.

Core commands

```
PYTHONPATH=src python -m unittest discover -s tests -v
PYTHONPATH=src python -m ugts_chess info
PYTHONPATH=src python -m ugts_chess perft --depth 4
PYTHONPATH=src python -m ugts_chess solve --depth 4 --fen "8/8/8/8/k7/8/1QK5 w - - 0 1"
PYTHONPATH=src python -m ugts_chess prove-mate --plies 3 --fen "8/8/8/8/k7/8/1QK5 w - - 0 1" --out
examples/mate_in_two_proof.json
PYTHONPATH=src python -m ugts_chess verify-proof examples/mate_in_two_proof.json
PYTHONPATH=src python -m ugts_chess probe --fen "8/8/8/8/k7/8/1QK5 w - - 0 1"
```

Included examples

Artifact	Purpose
mate_in_two_proof.json	Complete proof certificate with FEN/hash at every node.
mate_in_two_search.json	Heuristic search result with PV, counts and timing.
kqk_probe.json	Exact tablebase WDL/DTM probe and best move.
ruy_lopez_replay.json	Initial state plus deterministic hash-chained move events.
project.json	Versioned batch project containing perft, proof, tablebase and search tasks.
proof_viewer.html	Single-file offline viewer for a pasted or built-in proof certificate.
ugts_kc_chess-1.0.0-py3-none-any.whl	Installable wheel containing runtime and tablebase resources.

Regenerating the exact tables

```
PYTHONPATH=src python -m ugts_chess generate-tablebase --piece Q --out data/kqk.tb.gz
PYTHONPATH=src python -m ugts_chess generate-tablebase --piece R --out data/krk.tb.gz
```

Generation is deterministic: the gzip header uses mtime=0 and an empty filename; metadata stores the resulting SHA-256. Full regeneration is computationally heavier than probing but remains practical for these three-piece spaces.

11. Validation and Release Evidence

The release gates test legality, proof structure, exact state-space accounting, schemas, packaging and deterministic artifacts.

Legal-Move Regression: All 19 Perfth Checks Match

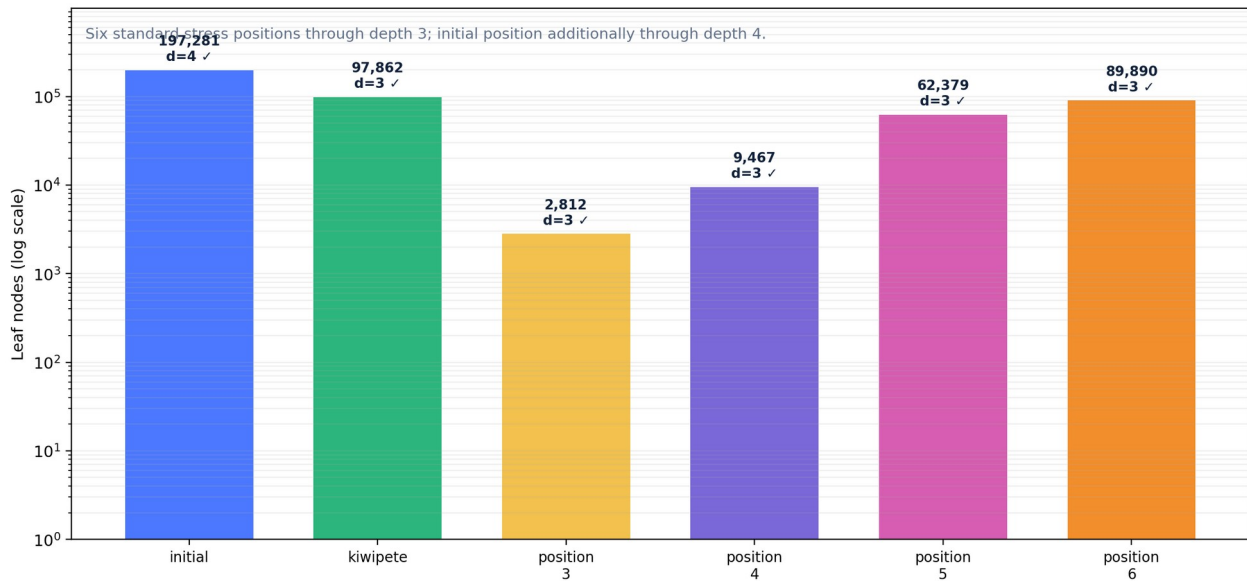


Figure 9. Legal-move regression. All 19 expected perfth counts match exactly.

11.1 Perfth coverage

Fixture	Depths	Expected = actual nodes
Initial position	Depth 1-4	20; 400; 8,902; 197,281
Kiwipete	Depth 1-3	48; 2,039; 97,862
Position 3	Depth 1-3	14; 191; 2,812
Position 4	Depth 1-3	6; 264; 9,467
Position 5	Depth 1-3	44; 1,486; 62,379
Position 6	Depth 1-3	46; 2,079; 89,890

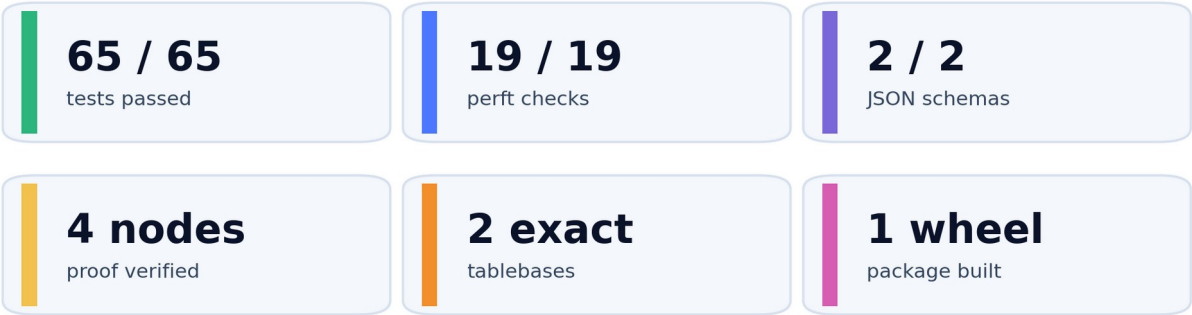
Why perfth matters. Perfth does not test playing strength, but it is a high-value structural oracle for move generation. These fixtures stress castling, en-passant, promotions, pins, checks and complex occupancy interactions.

11.2. Release Gates and Captured Metrics

Passing tests establish internal consistency of this package; they do not establish full-game solution or cross-device performance.

Release Validation Dashboard

Captured on Python 3.13.5 in the reference environment; timings are not cross-device performance claims.



Mate benchmark: mate +2 | 1,991 total nodes (including quiescence) | 0.427 s

Figure 10. Release-gate summary generated from validation/summary.json.

Gate	Status	What is established
Python compile	PASS	All source and tool modules compile.
Automated tests	65/65 PASS	Captured suite completed in 9.06 s.
Perft	19/19 PASS	Largest exact leaf count 197,281.
Proof verification	PASS	Mate certificate schema valid; 4 nodes independently verified after 418-node proof search.
Tablebases	PASS	Two SHA-checked tables, 296,674 compressed bytes total; maxima 20/32 plies.
JSON Schema	2/2 PASS	Project and proof examples validate under Draft 2020-12.
Wheel build	PASS	ugts_kc_chess-1.0.0-py3-none-any.whl built without downloading dependencies.
Reference benchmark	MEASURED	Mate +2; 1,195 main nodes + 796 quiescence nodes in one Python 3.13.5 run.

MEASUREMENT BOUNDARY

Run time and nodes per second are environment-specific. The exact claims are the rule outputs, proof verification, state-space counts and hashes - not a generalized speed advantage over mature chess engines.

12. Mechanism Catalog C001-C064

The namespaced catalog adds 64 chess-application mechanisms without renumbering the supplied UGTS source catalogs.

Table 5. Catalog distribution by domain

Domain	Count	IDs	Representative mechanisms
Tablebase	8	C048-C058	KXX exact profile; Validity cell; Terminal initialization; ...
Proof	6	C042-C047	OR attacker node; AND defender node; Checkmate leaf; ...
Search	6	C036-C041	Alpha-beta negamax; Iterative deepening; Quiescence; ...
State	5	C003-C009	Position tuple; Side-to-move bit; Castling-right mask; ...
Terminal	4	C028-C031	Checkmate relation; Stalemate relation; Dead-position subset; ...
Guard	3	C018-C020	King-safety guard; Castling transit guard; En-passant exposure guard
Transition	3	C023-C025	Deterministic board patch; Rights patch; Clock and turn patch
Encoding	2	C010-C034	Strict FEN codec; SAN projection
Event	2	C022-C026	Move proposal; Move event record
Identity	2	C005-C014	Piece identity; Repetition identity
Persistence	2	C011-C012	Canonical state record; SHA-256 state identity
Validation	2	C033-C063	Perft oracle; Automated release gates
Branch	1	C021	Promotion branch
Compatibility	1	C016	Occupancy and color gate
Compression	1	C056	Gzip table transport
Coordinates	1	C004	Square address
Evidence	1	C059	Tablebase metadata
Governance	1	C002	Evidence boundary
Heuristic	1	C035	Transparent evaluation
Indexing	1	C013	Compact 64-bit cache key
Packing	1	C049	Dense 19-bit KXX key
Presentation	1	C062	Offline proof viewer
Profile	1	C001	UGTS chess solver profile
Query	1	C032	Legal move enumeration
Relation	1	C017	Attack relation
Replay	1	C027	Hash-checked replay
Roadmap	1	C064	Distributed global proof sharding
Schema	1	C061	JSON contracts
Support	1	C015	Piece movement support
Symmetry	1	C057	Black-strong normalization
Tooling	1	C060	Command-line workflow

12.1 Representative operator records

ID	Domain	Mechanism	Normalized definition	Validation
C001	Profile	UGTS chess solver profile	A versioned application profile mapping standard	Implemented

ID	Domain	Mechanism	Normalized definition	Validation
			chess into typed state, verified moves, deterministic transitions, proof search and exact bounded tablebases.	
C009	State	Move counters	Halfmove and fullmove counters remain explicit for draw semantics and FEN round trips.	Implemented
C016	Compatibility	Occupancy and color gate	A supported destination must agree with side, own occupancy, capture and promotion semantics.	Implemented
C025	Transition	Clock and turn patch	Every move updates en-passant, halfmove, fullmove and side-to-move fields in a fixed order.	Implemented
C032	Query	Legal move enumeration	Generate all and only moves passing support, compatibility and king-safety guards.	Validated
C038	Search	Quiescence	At the horizon, captures, promotions and all check evasions extend tactical stability.	Implemented
C047	Proof	Finite-horizon status	Failure is reported as not forced within the declared horizon, never as a global game result.	Enforced
C055	Tablebase	Major-piece capture draw	Weak-king capture of the queen/rook exits to exact K-v-K draw.	Implemented
C064	Roadmap	Distributed global proof sharding	Shard by typed material/history state and admit only checker-verified boundary certificates.	Specified

CATALOG MEANING

Mechanism IDs are traceability entries for implementation and validation coverage. They are not, by themselves, claims of novelty, patentability, scientific priority, ownership or complete formal verification.

13. Boundaries, Kill Criteria and Final Definition

The profile is promoted only where its exact contracts outperform or complement simpler conventional approaches at equal correctness.

13.1 Explicit non-claims

- No weak, strong or ultra-weak solution of the standard initial chess position is claimed.
- No playing-strength comparison with Stockfish, Leela Chess Zero or another mature engine is claimed.
- No universal cross-platform bit-identical floating-point evaluation is claimed; legality and serialized proof/hash records are the exact boundary.
- No complete 4- through 7-piece tablebase corpus is bundled; only complete KQK and KRK DTM tables are included.
- No exhaustive formal proof of the Python interpreter, operating system or hardware is claimed.
- No legal identity, authorship, patentability, licensing or ownership inference follows from the requester attribution.

13.2 Kill criteria

- Any legal-move mismatch appears in perft, SAN round trips or special-rule tests.
- A proof verifier can accept a missing defender reply, illegal move, bad FEN/hash or non-mate leaf.
- A tablebase address collision occurs, counts fail to sum to 2^{19} , or sampled retrograde relations fail.
- Compact keys, gzip size or cache behavior are promoted as correctness without reconstruction and collision/error contracts.
- Repetition history, claim policy or halfmove thresholds are implicit in a result that depends on them.
- Distributed workers can mutate authoritative state or promote a claim without independent certificate verification.
- The UGTS layer becomes more complex, slower or less auditable than a conventional rules/search/tablebase implementation at equal error and evidence.

13.3 Final formal definition

FORMAL DEFINITION

UGTS-KC Chess 1.0 is a finite, content-addressed, deterministic application profile in which a typed chess position and history define move supports; compatibility and special-rule guards filter proposals; king-safety evaluation certifies legal events; atomic transitions patch board, rights, en-passant state and clocks; SHA-256 lineage supports replay; exact OR/AND certificates prove forced mate within a declared horizon; complete KQK/KRK retrograde tables answer bounded endgames; and heuristic alpha-beta search remains a downstream estimator.

Canonical shorthand: position + history -> move support -> compatibility -> special-rule and king-safety guards -> verified move event -> deterministic state patch -> hash lineage/replay -> optional search, proof, tablebase and presentation outputs.

Appendix A. Package Map and Reproduction

The ZIP contains the complete source, exact data, examples, report source and captured evidence needed to inspect or rerun the release.

Path	Contents
src/ugts_chess/	Dependency-free rule kernel, events/replay, search, proofs, tablebase generator/prober and CLI.
data/	KQK and KRK compressed tables plus metadata and SHA-256.
spec/	Formal definition, 64-mechanism catalog, JSON schemas and source register.
examples/	Mate proof/search/probe, replay, project and runnable demo.
tests/	65 standard-library unittest cases.
validation/	Compile, tests, perft, schemas, tablebase hashes, benchmark, wheel build and release summary.
web/	Self-contained HTML proof viewer.
report/	This PDF, editable DOCX source and generated figures.
checksums/	Final package SHA-256 manifest.

Clean reproduction sequence

```
cd UGTS_KC_CHESS_1_0
PYTHONPATH=src python tools/generate_examples.py
PYTHONPATH=src python tools/run_validation.py
python tools/generate_report_assets.py
python tools/build_report.py
```

Wheel installation. The built wheel is under validation/dist/. After installation, the console entry point is ugts-chess and the tablebase resources load from the installed package.

Release identifiers

Identifier	Value
Runtime version	1.0.0
Solver schema	ugts-kc-chess-solver-1.0
Proof schema	ugts-kc-chess-mate-proof-1.0
Tablebase schema	ugts-kc-chess-kxk-tablebase-1.0
Catalog range	C001-C064
Automated tests	65
Exact tablebases	KQK and KRK

Appendix B. Source Register and References

Supplied sources are referenced by exact SHA-256 and are not redistributed inside the accompanying ZIP.

Source	Page basis	SHA-256 prefix	Role
UGTS-GN 1.1	pp. 2-5, 9-15	3a6b6c57118ca4e2d7...	Canonical support, compatibility, guard, event, transition, lineage; precision and compression boundaries
UGTS-KC 2.0	pp. 3-5, 8-15	a29dd9959ebd300c32...	Typed state, query-first patterns, topology, hybrid events, bounded dynamics
KC Two Hands 3.0	pp. 5-6, 11-16, 19-20	183ffa4e2bf29cfa03...	Proposal verification, deterministic commit, state hashes, checkpoints and replay
UGTS-KC 3.6	pp. 3-7, 9-14	e21433cd8378368daf...	Definitions as data, content addresses, dependency closure and explicit operation order
UGTS-KC 3.6.2 SCLP	pp. 7-15	6987490a0a7b7ed76b...	Finite packed keys, radix tries, bounded branching and storage audits
UGTS-KC 3.9	pp. 3-4, 8-13	d7ee3c91abf60e35b4...	Deterministic game world, canonical snapshots/hashes, CLI and offline HTML5 delivery
UGTS-KC 3.6.3 SARA	pp. 1-2, 6-12	c5a8256e345047d56f...	Reviewed but excluded as wallet/security-specific; only its explicit authorization boundary is retained as a governance analogy

External standards/context

FIDE Laws of Chess taking effect from 1 January 2023. Official handbook:

<https://handbook.fide.com/chapter/e012023>. Used for current rule terminology and draw/checkmate semantics; the implementation boundary is described in Section 4.

Syzygy endgame tablebases. Public context: <https://syzygy-tables.info/>. Used only to state that public exact endgame coverage extends beyond the two bundled three-piece tables; no Syzygy files or code are redistributed.

Requester and legal boundary

Prepared for Tom Klootwijk. The requester attribution is recorded as supplied and has not been independently verified. This report and package are technical design artifacts. They are not legal proof of identity, authorship, ownership, patentability, priority, exclusive rights, licensing status or chain of title.

Closing statement

RELEASE INTERPRETATION

The package provides an auditable path from chess state to legal move, verified event, deterministic replay, bounded mate proof and exact finite endgame result. Its strongest claim is not that all chess has been solved, but that each promoted result states exactly what was solved, under which profile, with which certificate, hash, test and boundary.